Instituto de Informática — Universidade Federal do Rio Grande do Sul (UFRGS)

Análise do Impacto de Runtimes Task-Based no Desempenho de Algoritmos de Álgebra Linear Densa

Matheus Augusto Tregnago, Lucas Mello Schnorr, Philippe O. A. Navaux

{matregnago,navaux,schnorr}@inf.ufrgs.br

SSCAD 2026

Contexto e motivação

- Plataformas de HPC cada vez mais heterogêneas
- Runtimes *task-based* no modelo **Sequential Task Flow**: a aplicação vira um DAG de tarefas, escalonado dinamicamente sobre os recursos
- **Chameleon**: álgebra linear densa por blocos que delega a execução a um runtime externo
  - troca de runtime sem reescrever o algoritmo
- **StarPU** e **PaRSEC**: runtimes suportados pelo Chameleon, com estratégias de escalonamento diferentes

Trabalhos relacionados

Trabalho	O que comparam	Limitação
Hoque et al. (2017)	PTG × DTD no PaRSEC; Chameleon com PaRSEC, StarPU e QUARK	somente CPU
Lacoste et al. (2014)	StarPU × PaRSEC no solver PaStiX, com GPU	álgebra esparsa: DAG irregular
Agullo et al. (2015)	Cholesky no StarPU × limites teóricos	um único runtime
Lisito et al. (2025)	LU com pivotamento: StarPU + Chameleon × PaRSEC + DPLASMA	biblioteca diferente por runtime

> Lacuna: nenhum compara StarPU e PaRSEC em álgebra densa, com a mesma biblioteca, em plataforma heterogênea

StarPU × PaRSEC

StarPU

- Modelo STF: DAG desenrolado tarefa a tarefa
- **dmda**: modelo de desempenho aprendido do histórico + custo de transferência. Usa o recurso com menor término estimado
- **dmdas**: dmda + prioridade pelo caminho crítico do DAG

PaRSEC

- Interface DTD: inserção sequencial, dependências inferidas
- **lfq**: filas locais por worker + roubo de tarefas
- **gd**: uma única fila global, sem localidade
- Escolha CPU/GPU estática, sem modelo de desempenho

> Expectativa: O StarPU deve apresentar um desempenho superior ao PaRSEC, por conta dos modelos de performance.

Contribuições

Dificuldade encontrada: A interface DTD do PaRSEC não executava tarefas em GPU e os codelets do Chameleon para o PaRSEC eram só CPU.

- **PaRSEC (DTD)**: nova função de inserção com implementação CPU e GPU. A escolha do dispositivo reaproveita a heurística da interface PTG
- **PaRSEC**: tarefa de CPU que escreve um bloco passa a invalidar as cópias na GPU
- **Chameleon**: codelets GPU para o PaRSEC com cuBLAS/cuSOLVER
- **Análise**: rastros do PaRSEC no StarVZ
- **Reprodutibilidade**: ambiente fixado com Nix

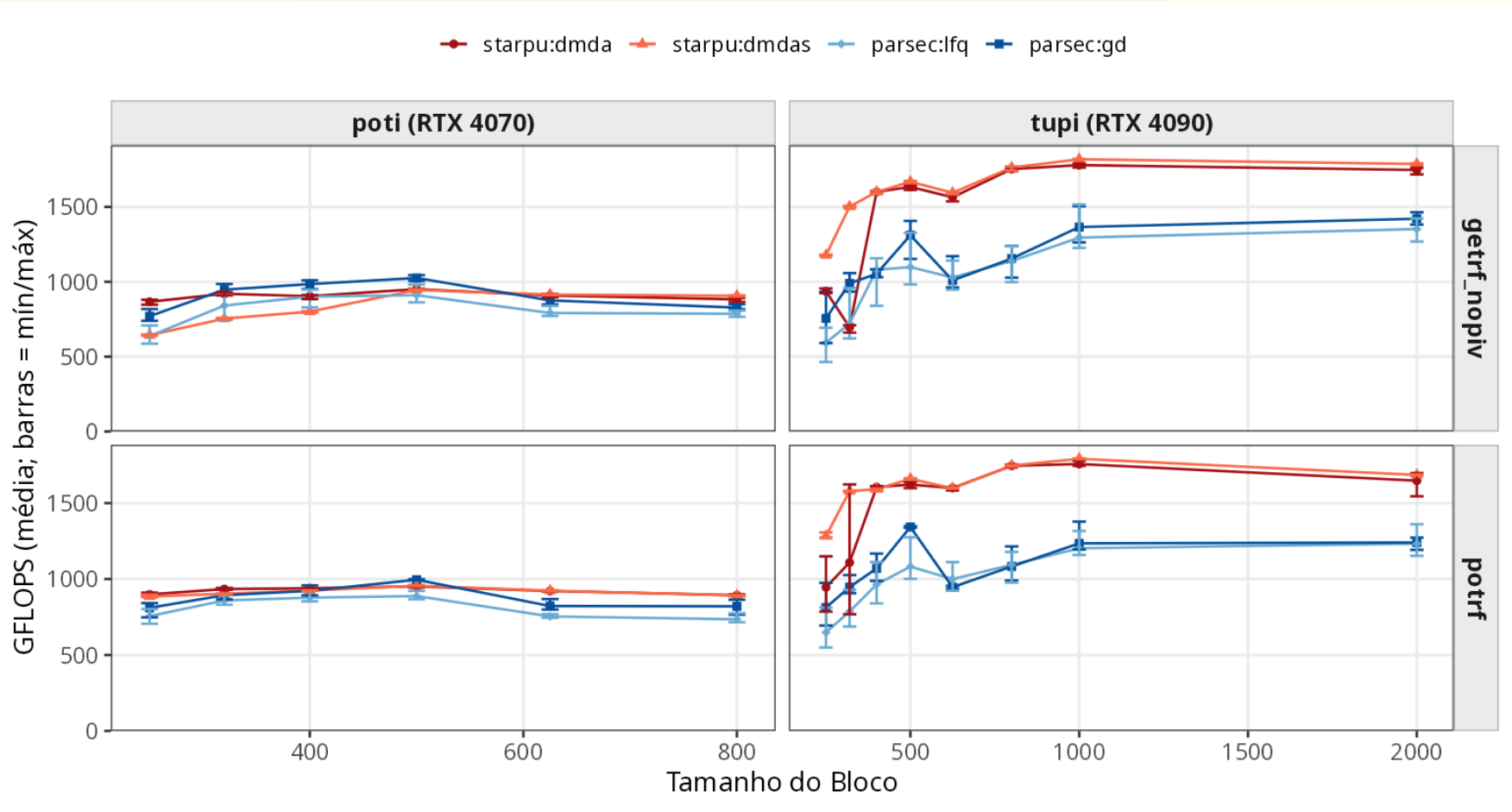
Metodologia experimental

	poti	tupi
CPU	i7-14700KF, 20 núcleos (8 P-cores)	i9-14900KF, 24 núcleos (8 P-cores)
RAM	96 GB	128 GB
GPU	RTX 4070, 12 GB	RTX 4090, 24 GB
Pico FP64 da GPU	**455 GFLOPS**	**1.290 GFLOPS**

- Cholesky (**potrf**) e LU sem pivotamento (**getrf_nopiv**) em FP64

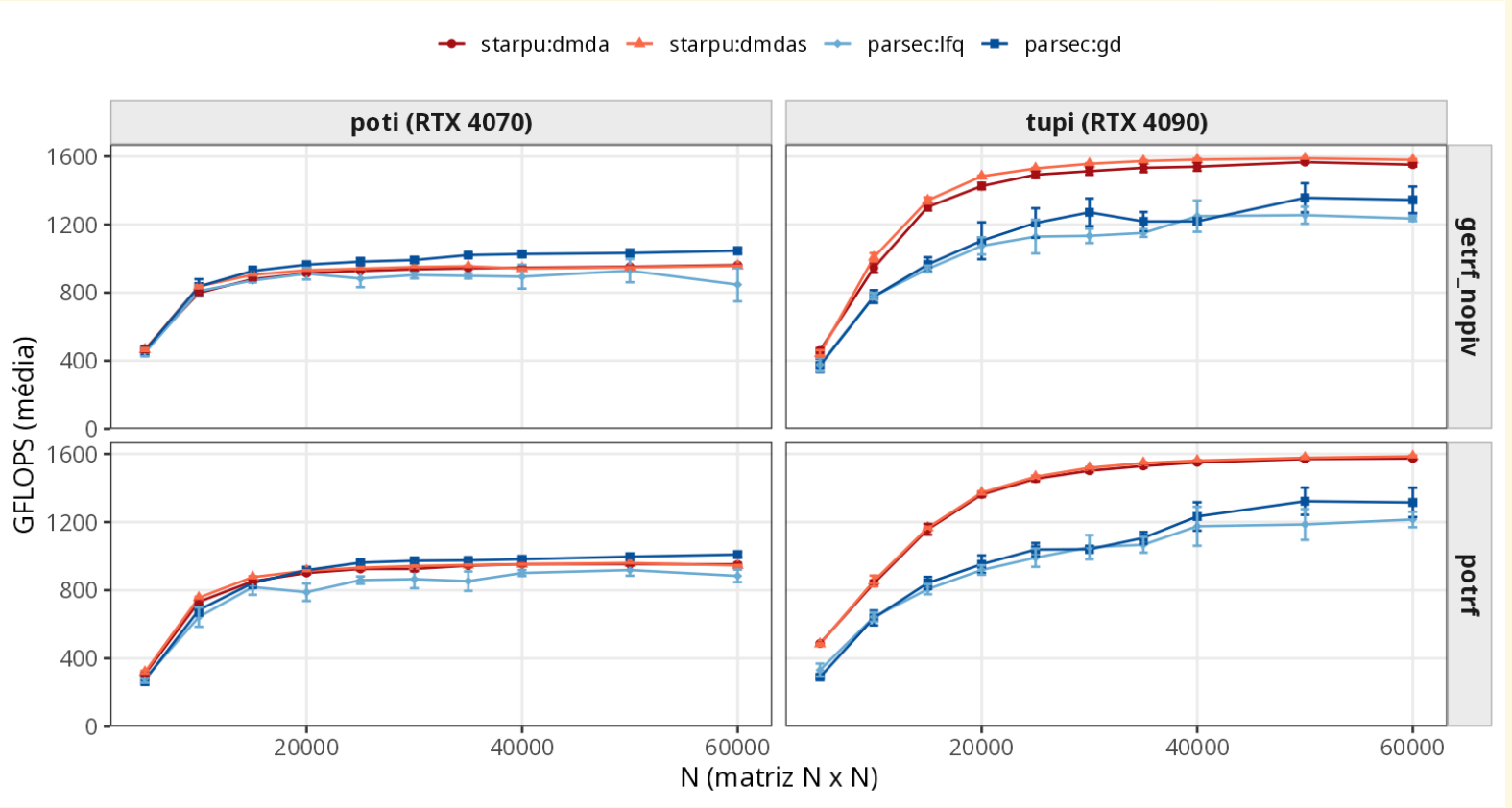
1) varredura do tamanho de bloco b com $N = 40.000$
2) fatorial completo: máquina × configuração (4) × N (10 valores, 5.000 a 60.000), ordem aleatória, 5 repetições
3) rastros de execução analisados com StarVZ

Escolha do tamanho de bloco (*N* = 40.000)



Bloco ótimo: ***b* = 500** na poti e ***b* = 1.000** na tupi

Desempenho em função de *N*



poti: `parsec:gd` +4–8% ($N \geq 20.000$) · **tupi**: StarPU sempre à frente (~30% em $N = 40.000$)
→ contraria a expectativa: o vencedor **depende da máquina**

Onde está a diferença?

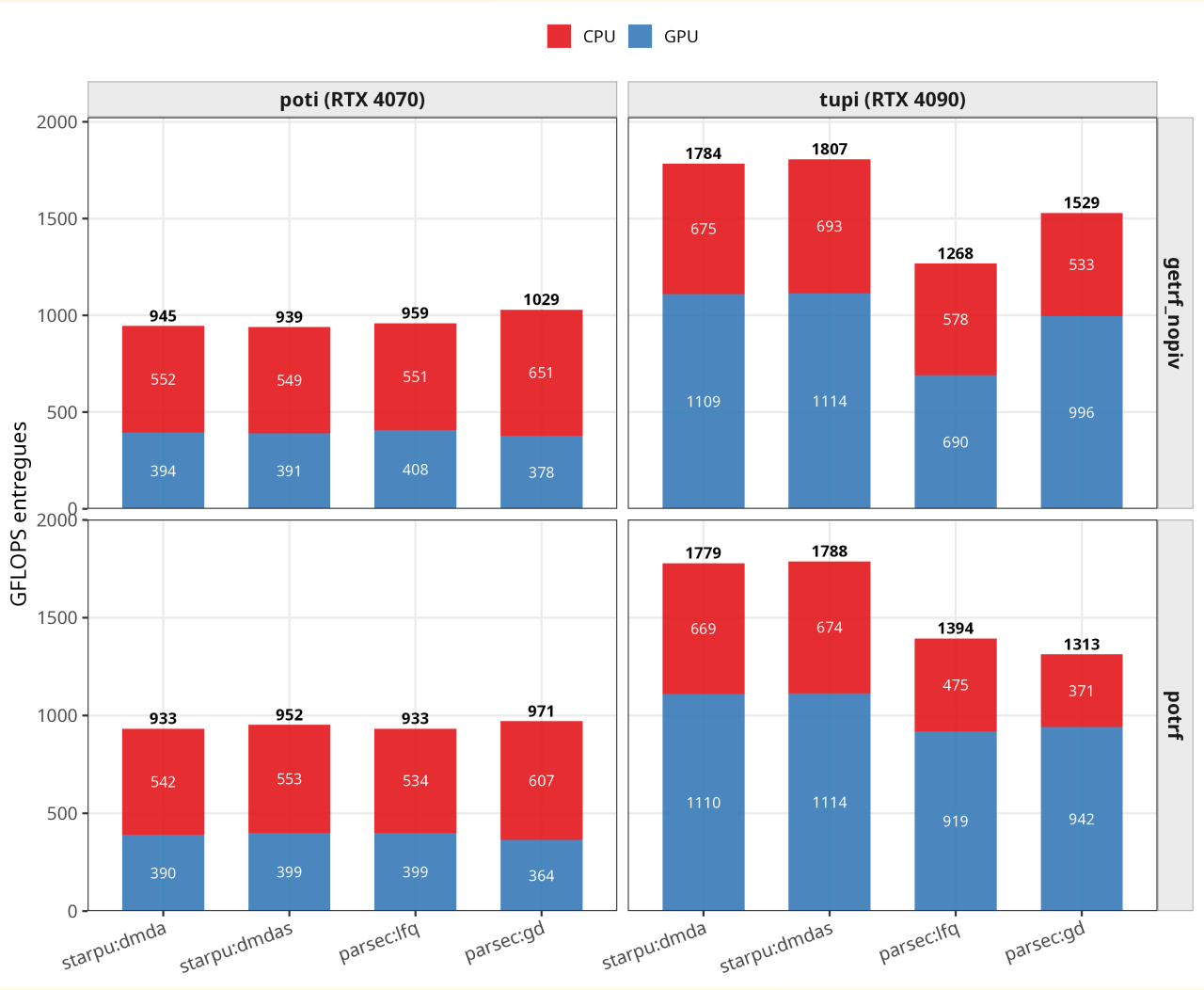
GFLOPS entregues por CPU e por GPU,
$N = 40.000$

poti (RTX 4070)

- GPU quase igual nas 4 configurações (364–399 GFLOPS na Cholesky)
- `parsec:gd` ganha **na CPU**: 607 × 542 GFLOPS do `starpu:dmda`
- StarPU reserva **1 thread por worker CUDA**; PaRSEC despacha a GPU a partir do próprio worker

tupi (RTX 4090)

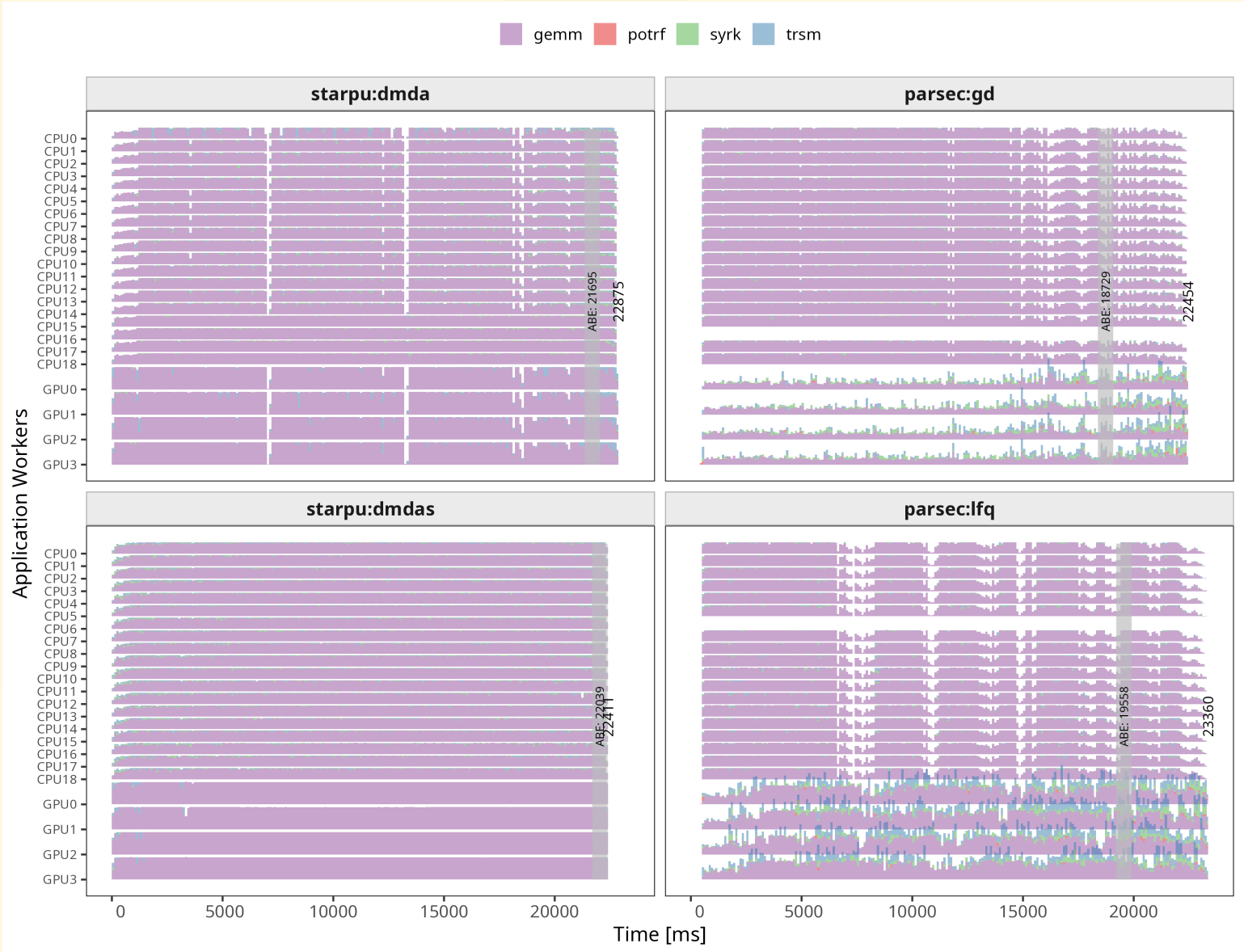
- GPU domina: >1.100 GFLOPS no StarPU × 942 no `parsec:gd`
- CPUs do `parsec:gd` ociosas em **~50%** do makespan (StarPU: <10%)



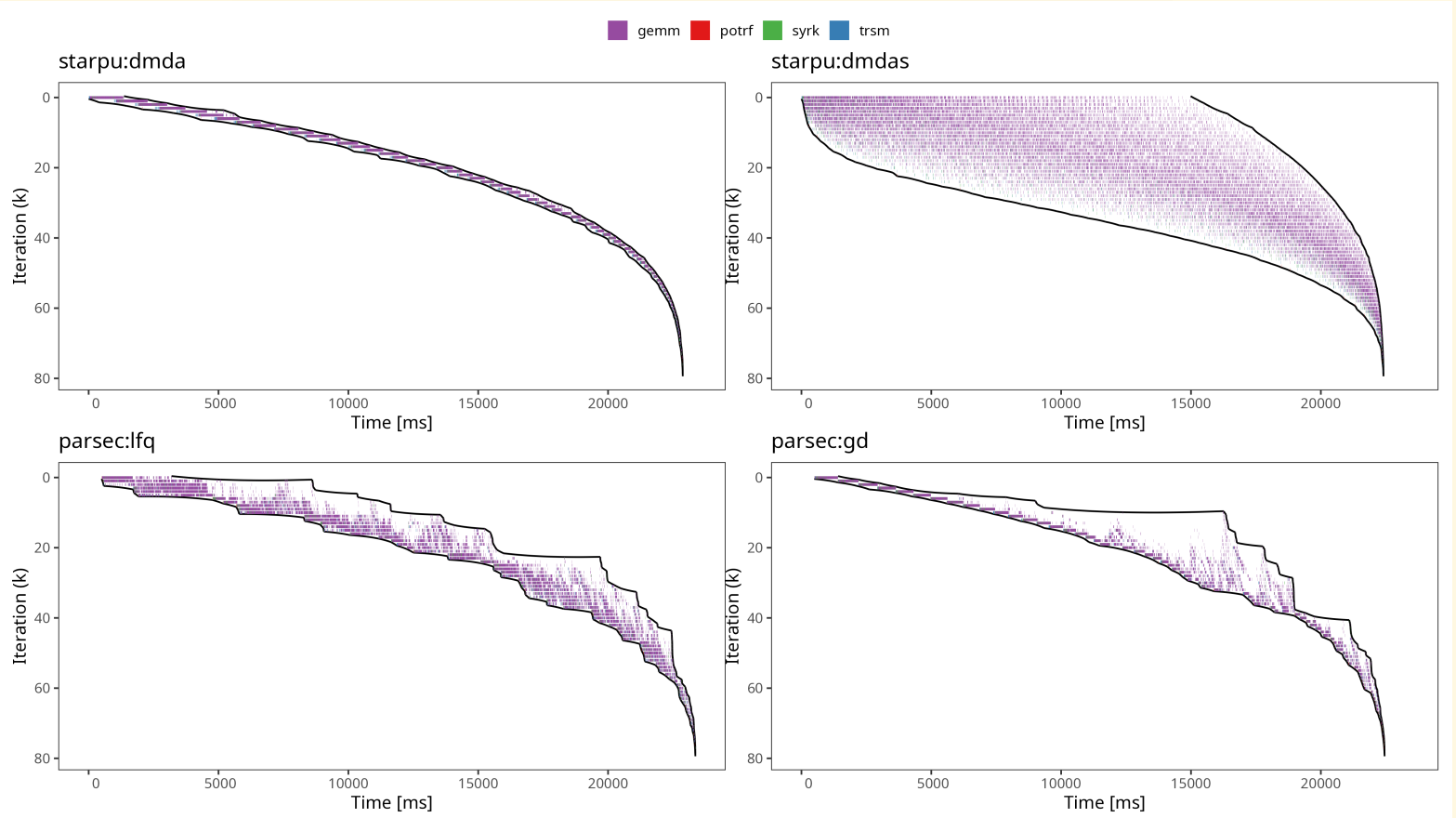
Espaço-tempo

Cholesky na poti, $N = 40.000$, $b = 500$

- Linhas: 19 workers de CPU + 4 streams de GPU; cor = tipo de tarefa
- StarPU e `parsec:gd`: CPUs saturadas, **1,6–8,7%** de ociosidade
- `parsec:lfq`: lacunas recorrentes, **13,9%** de ociosidade → menor parcela de CPU (534 GFLOPS)



Progressão das iterações (Cholesky, poti)



Iterações simultâneas em média: `dmda` **1,9** · `dmdas` **28,7** (adianta o caminho crítico) · PaRSEC **7,7** e **9,3** (tarefas retardatárias retidas nas filas)

Conclusões

- StarPU é **superior ou equivalente** na maior parte dos casos
- A exceção depende do **equilíbrio CPU × GPU** da máquina:
  - **poti**: GPU ≈ metade da CPU → `parsec:gd` +4–8%, por não reservar threads de CPU para a GPU
  - **tupi**: GPU ≈ 2× a CPU → StarPU ~30% melhor, extraindo mais da GPU
- Não decide a eficiência na GPU, e sim **quanto sobra de CPU**

Trabalhos futuros

- Medir a duração das tarefas de GPU no próprio dispositivo no PaRSEC
- Mais combinações de CPU e GPU, para achar o limiar da inversão
- Ambientes multi-nó e multi-GPU
- Suporte a GPU na DTD para outras fatorações (QR)

Referências

1. Augonnet, C. et al. "StarPU: a unified platform for task scheduling on heterogeneous multicore architectures." *Concurrency and Computation: Practice and Experience*, 23(2), 2011.
2. Bosilca, G. et al. "DAGuE: A Generic Distributed DAG Engine for High Performance Computing." *IPDPS Workshops*, 2011.
3. Agullo, E. et al. "A Hybridization Methodology for High-Performance Linear Algebra Software for GPUs." *GPU Computing Gems Jade Edition*, 2012.
4. Hoque, R. et al. "Dynamic Task Discovery in PaRSEC: A Data-flow Task-based Runtime." *ScalA*, 2017.
5. Lacoste, X. et al. "Taking Advantage of Hybrid Systems for Sparse Direct Solvers via Task-Based Runtimes." *IPDPS Workshops*, 2014.
6. Agullo, E. et al. "Bridging the Gap between Performance and Bounds of Cholesky Factorization on Heterogeneous Platforms." *IPDPS Workshops*, 2015.
7. Lisito, A. et al. "Scalable and Portable LU Factorization with Partial Pivoting on Top of Runtime Systems." *IPDPS*, 2025.
8. Pinto, V. G. et al. "Providing In-depth Performance Analysis for Heterogeneous Task-based Applications with StarVZ." *IPDPS Workshops*, 2021.

Agradecimentos

- **PCAD** — Parque Computacional de Alto Desempenho do INF/UFRGS, onde foram realizados todos os experimentos (pcad.inf.ufrgs.br)
- **Petrobras** — financiamento parcial, por meio de acordo com a UFRGS (projetos 2020/00182-5 e 2018/00263-5)
- **CAPES** — Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (Código de Financiamento 001)

Instituto de Informática — Universidade Federal do Rio Grande do Sul (UFRGS)

Obrigado!

Matheus Augusto Tregnago, Lucas Mello Schnorr, Philippe O. A. Navaux

{matregnago,schnorr,navaux}@inf.ufrgs.br

github.com/matregnago/chameleon-runtime-benchmarks/tree/sscad2026